

ORCHESTRA: An Object-Oriented Framework for Implementing Chemical Equilibrium Models

JOHANNES C. L. MEEUSSEN*

Soil Science Group, Macaulay Institute, Craigiebuckler,
Aberdeen, AB15 8QH, U.K.

This work presents a new object-oriented structure for chemical equilibrium calculations that is used in the modeling framework ORCHESTRA (Objects Representing CHEmical Speciation and TRANsport). In contrast to standard chemical equilibrium algorithms, such as MINEQL, MINTEQA2, PHREEQC, and ECOSAT, model equations are not hard-coded in the source code, but instead all equations are defined in text format and read by the ORCHESTRA calculation kernel at run time. This makes model definitions easily accessible and extendible by users. Furthermore, it results in a very compact and efficient calculation kernel that is easy to use as a submodel within mass transport or kinetic models. Finally, the object-oriented structure of the chemical model definitions makes it possible to implement a new object-oriented framework for implementing chemical models. This framework consists of three basic object types, entities, reactions, and phases, that form the building blocks from which other chemical models are composed. The hierarchical approach ensures consistent and compact model definitions and is illustrated here by discussing the implementation of a number of commonly used chemical models such as aqueous complexation, activity correction, precipitation, surface complexation ion exchange, and several more sophisticated adsorption models including electrostatic interactions, NICA, and CD-MUSIC. The ORCHESTRA framework is electronically available from www.macaulay.ac.uk/ORCHESTRA.

Introduction

Chemical equilibrium, or speciation, models are used to calculate the distribution of chemical substances over different physicochemical forms and phases in a chemical equilibrium system. Such models are widely used to describe chemical processes that occur in soils and in the aqueous environment. As chemical speciation strongly affects the mobility and chemical behavior of substances, speciation algorithms are also often used as subroutines in mass transport and kinetic models. There is a range of well-established programs available in this area, such as MINEQL (1), MINTEQA2 (2), PHREEQC (3), GWB (4), and ECOSAT (5). These programs are all set up as general purpose chemical speciation tools that allow users to define a specific chemical equilibrium model system in terms of predefined model elements such as “master components”, “species”, “gases”, and “surfaces”.

These programs allow the set of chemical reactions, including reaction constants and stoichiometry coefficients, to be defined as input, but in all cases, the actual equations that make up the different chemical models are defined in the program source code. This makes it impossible for users to change or add model definitions without changing the source code. Changing the source code is required to implement more advanced adsorption models for organic surfaces such as Tipping's MODEL V (6) that uses discrete sites or the NICA model (7, 8) that uses sites with an affinity distribution. Such heterogeneous *K* models are reported to be very difficult to implement within the standard structure of current speciation models (9). Changes in the source code are also required to implement some models for adsorption of ions at oxide surfaces such as the CD-MUSIC model (10). Even when source code is available, adding new model code to existing algorithms is not straightforward because the model definition code is usually intimately intertwined with the rest of the program code. As a result adsorption models that cannot be set up in input files of existing programs are often implemented as dedicated programs (6).

Another approach is to implement such models within the existing structure of speciation algorithms but then as an exception to the standard matrix calculations. This requires additional dedicated code that handles these special cases. Although it is certainly possible to implement a wide range of models in this way, as demonstrated by ECOSAT (5), which contains a large selection of the latest adsorption models, when these exceptions become a rule rather than an exception, adhering to the standard matrix approach can become a limitation.

For these reasons, a fundamentally different software approach was developed resulting in the ORCHESTRA (Objects Representing CHEmical Speciation and TRANsport) modeling framework described here. The most significant difference from any of the standard speciation algorithms is that in ORCHESTRA the model-type definitions are completely separated from the calculation kernel and even from the source code. Model types are defined in the form of objects, in text format. First, this makes ORCHESTRA flexible and transparent, as it allows users to freely change or add model definitions without changing or recompiling the source code. Second, because the calculation kernel does not contain any information on chemical models, it has become extremely small and efficient. Finally, the object-oriented structure of the model-type definitions made it possible to design a generic object framework, based on a small set of fundamental object classes, that can be used as building blocks for specific model implementations. This greatly simplifies the implementation of new chemical models and secures the consistency between different models, which is illustrated here by discussing a number of examples.

Structure of the Chemical Equilibrium Framework

The ORCHESTRA equilibrium framework consists of two separate parts (Figure 1): (1) a generic equation solver, in the form of an executable (Java) program; (2) a set of object-oriented (chemical) model definitions in text format that is read by the equation solver as input at run time.

The combination of the equation solver and the object definitions can be used in a similar way to a standard speciation program.

Equation Solver

The ORCHESTRA equation solver is completely generic and can be “programmed” in an input file by defining the

* Corresponding author. Present address: Energy Research Centre of the Netherlands (ECN), Westerduinweg 3, P.O. Box 1, 1755 ZG, Petten, The Netherlands. Phone: +31 0224 564006. Fax: +31 0224 568163. E-mail: meeussen@ecn.nl.

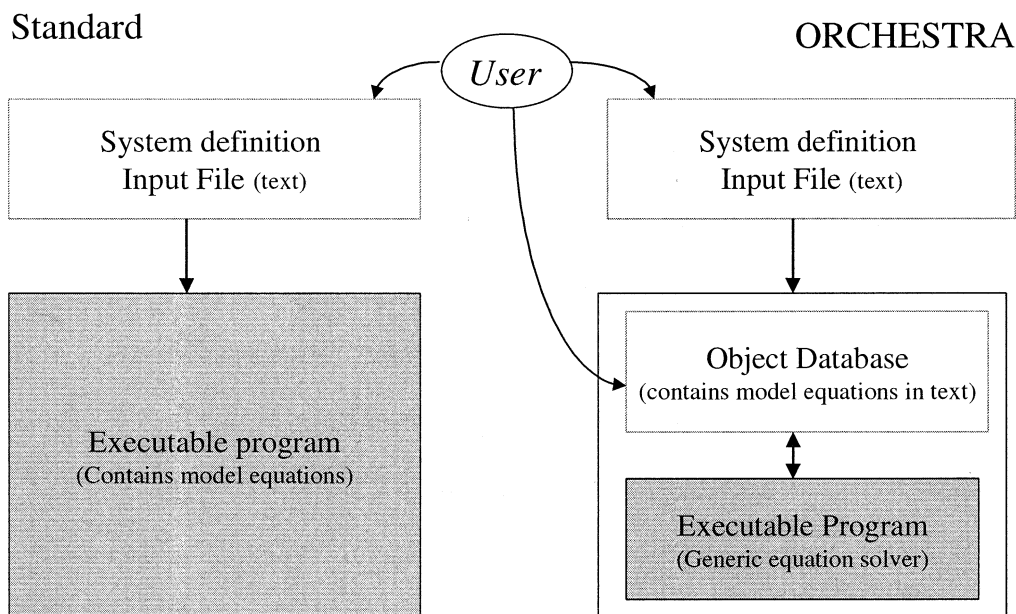


FIGURE 1. Overview of the structure of the ORCHESTRA framework in comparison with the structure that is used in standard speciation programs.

following: (1) numerical variables; (2) literal calculations in the form of mathematical expressions; (3) pairs of variables as unknown-equation pairs; (4) new “objects” in terms of literal calculations and other objects.

With these four native language elements, the equation solver can be programmed to perform arbitrary calculations. In fact, kinetic and transport calculations in ORCHESTRA are also carried out by (different copies of) the same equation solver. For transport calculations, the equation solver has access to the state variables of two connected cells.

Defining Variables. The definition of a variable “pH” with a default value of 3.75, is as follows:

@Var: pH 3.75

The pH can now be used in expressions, as is shown next.

Defining Mathematical Expressions. Similar to standard computer languages, mathematical expressions can be defined in terms of variables, operators, and a set of standard mathematical functions including conditional expressions, which can be used as a form of “if” statement. Different from standard computer languages is the requirement that each calculation is assigned to take place during one of four calculation stages.

For example, the proton activity can be calculated from the pH during stage 1 through the statement

@Calc(1, “H+.act = 10 ^ -pH”)

The four different calculation stages and the type of calculations performed are as follows:

Stage 1: Calculation of activities (or intensities, such as surface potentials or Boltzmann factors) of all chemical entities. After stage 1, all activities of chemical entities are known and available for the calculations during the later stages. Calculations during stage 1 are carried out in the same order as they are added to this stage.

Stage 2: Calculation of the concentrations of all chemical entities using the activities and ion activity correction models. After stage 2, all entity concentrations in their own system phase are calculated. The stage 2 calculations are carried out in the same order as they are added to this stage.

Stage 3: Calculation of cumulative mass balances of chemical entities per individual system phase using the entity

concentrations calculated in stage 2. The stage 3 calculations are carried out in the reverse order from the order in which they are added to this stage. The last ones added are carried out first.

Stage 4: Calculation of cumulative mass balances over all phases using the calculated local mass balances and the phase hierarchy. Calculations within this stage are also carried out in the reverse order from the order in which they are added to this stage. The last ones added are carried out first.

This ordering of calculations per stage and within the stages is essential for the hierarchical structure of entities and system phases, as is shown later.

Defining Unknown-Equation Pairs. Pairs of variables can be selected as unknown-equation pairs. The value of the unknown variable is then estimated by a Newton–Raphson iteration procedure, and all calculations are performed repeatedly until the calculated value of the equation variable equals (or is close to) a given value.

An example of an unknown-equation pair definition, is as follows:

@UnEq:(pH, H+.tot.sum, 1e-3)

In this case, the pH is selected as unknown, which is iteratively updated until the accompanying equation value H+.tot.sum (total proton concentration) equals 1e-3 (M). In contrast to most other chemical speciation algorithms, in ORCHESTRA, the Jacobian with the partial derivatives of unknowns to equation functions is determined numerically. This has the advantage that no analytical expressions for the Jacobian elements are required in the source code and that the iteration procedure can be used for arbitrary unknown-equation combinations that are defined in the input file. A potential disadvantage of calculating derivatives numerically is that this could be slower than using analytical expressions. With the ORCHESTRA calculation kernel, this is not necessarily the case. When a single input variable is changed, as is repeatedly done during this process, only those variables that are affected by this change are recalculated.

As discussed later, the ability to select arbitrary unknown-equation pairs can be very useful in the definition of adsorption models. This feature is also useful for implementing ion activity correction – ionic strength calculations. For example, the Davies equation implementation that is

included in the set of standard ORCHESTRA objects uses an “unknown” variable that contains the estimated ionic strength. The difference between estimated and calculated (from chemical speciation) ionic strength is defined as an equation that should become zero. The same principle is used within the source code of PHREEQC, although the analytical expression for partial derivatives is used there. In ORCHESTRA, the complete definition of the Davies equation is present in text form and can be easily changed or exchanged for a different activity correction model.

Example of an Input File with Literal Calculations. The following example shows how the ORCHESTRA equation solver can be programmed to perform a chemical equilibrium calculation by literally writing out all the variables, expressions, and unknown-equation pairs involved. If we consider a system of two aqueous species, Cl^- and Cd^{2+} , that can react to form a third species, the complex CdCl^+ , with a formation constant of 95.5. This can be represented in the following tableau:

CdCl^+	Cd^{2+}	Cl^-	K
	1	1	95.5

Cd^{2+} and Cl^- are the components, or master species, for which the activities are directly given or estimated and CdCl^+ is a species that is formed from the components by the formation reaction:



This reaction equation implies that the following relationship exists between the activities of the species:

$$(\text{CdCl}^+) = K(\text{Cd}^{2+})^1(\text{Cl}^-)^1$$

The mass balances of the components are related as follows:

$$\text{mass balance } \text{Cd}^{2+} = 1[\text{Cd}^{2+}] + 1[\text{CdCl}^+]$$

$$\text{mass balance } \text{Cl}^- = 1[\text{Cl}^-] + 1[\text{CdCl}^+]$$

Next, the complete set of literal statements is given in an input file to calculate the composition of this system at a given total amount of chloride and cadmium. For reasons of simplicity, activity corrections are ignored.

Definition of Variables. For each species, three variables are defined to store its activity, concentration and mass balance:

```
@Var: Cd2+.1e-5
@Var: Cd2+.con 0
@Var: Cd2+.sum 1e-3

@Var: Cl-.act 1e-5
@Var: Cl-.con 0
@Var: Cl-.sum 1e-4

@Var: CdCl+.act 0
@Var: CdCl+.con 0
@Var: CdCl+.sum 0
```

The CdCl^+ species is formed by a reaction and therefore additionally has a formation constant:

```
@Var: CdCl+.k 95.5
```

Definition of Unknown-Equation Pairs. The system has an unknown-equation pair for each component. The un-

known is the activity of the component, while its total amount is chosen as the accompanying equation:

```
@UnEq:(Cd2+.act, Cd2+.sum, 1e-3)
```

```
@UnEq:(Cl-.act, Cl-.sum, 1e-4)
```

The initial values of the component activities are used as initial estimates in the numerical calculation, while the mass balances are defined by fixed given values.

Definition of Calculations. It is now necessary to define the expressions to calculate the equation values (mass balances of Cd^{2+} and Cl^-) from the selected unknowns (activities of Cd^{2+} and Cl^-) and input variables (CdCl^+ formation constant) during the different calculation stages. These are given by the following.

Stage 1, Activity Calculations. The activity of CdCl^+ is calculated from its K value and the activities of its parent species

$$\text{CdCl}^+.\text{act} = K(\text{Cd}^{2+}.\text{act})^1(\text{Cl}^+.\text{act})^1$$

```
@Calc:(1, "CdCl+.act = CdCl+.k")
```

```
@Calc:(1, "CdCl+.act = CdCl+.act * (Cd2+.act)^1")
```

```
@Calc:(1, "CdCl+.act = CdCl+.act * (Cl-.act)^1")
```

Stage 2, Calculate Concentrations from Activities and Initialize Mass Balances with Concentrations. Then, without ion activity corrections, the concentration of each entity can be set equal to its activity.

```
@Calc:(2, "Cd2+.con = Cd2+.act")
```

```
@Calc:(2, "Cl-.con = Cl-.act")
```

```
@Calc:(2, "CdCl+.con = CdCl+.act")
```

Each cumulative mass balance is then initialized with the concentration of the species:

```
@Calc:(2, "Cd2+.sum = Cd2+.con")
```

```
@Calc:(2, "Cl-.sum = Cl-.con")
```

```
@Calc:(2, "CdCl+.sum = CdCl+.con")
```

Stage 3, Calculate Cumulative Mass Balance Calculations. Finally, the cumulative mass balances of Cd^{2+} and Cl^- are calculated:

```
@Calc:(3, "Cd2+.sum = Cd2+.sum + CdCl+.sum * 1")
```

```
@Calc:(3, "Cl-.sum = Cl-.sum + CdCl+.sum * 1")
```

The complete set of statements given here will repeatedly estimate the values of the “unknown” cadmium and chloride activity until the calculated values of their mass balances equal (within a convergence criterion) the given values.

Defining Objects. Although the approach outlined above is very flexible, it is of course impractical to implement complicated equilibrium systems by literally writing out all the calculations involved. This problem is solved by making it possible to define so-called “object classes”. These object classes act as named templates that can contain generic calculation recipes. It is now illustrated how the literal calculations above can be converted into a number of object classes.

If the literal code given above is analyzed, it is clear that the calculations involved in the three chemical species Cd^{2+} , Cl^- and CdCl^+ are similar and therefore suitable for conversion into an object class “species”. The species name can be used as a parameter. This results in the following object class definition that could be written either directly in the input

file or in a separate file, for example myobjects.txt, that can be included in the input file:

```
@Class: species(name){
  @Var: <name>.act 0
  @Var: <name>.con 0
  @Var: <name>.sum 0
  @Calc:(2, "<name>.con = <name>.act ")
  @Calc:(2, "<name>.sum = <name>.con ")
}
```

When this object is used in the input file to define species objects, the placeholder <name> is substituted with the actual name:

```
@species(CdCl+)
```

In a similar way, an object class can be defined that represents a primary species or component with a given total mass balance:

```
@Class: component(name, given__mass__balance){
  @species(<name>)
  @UnEq:(<name>.act, <name>.sum,
        <given__mass__balance>)
}
```

Note that this object class can be quite small because it is defined in terms of the species class. In the input file, the components object can be used as follows:

```
@component(Cl-, 1e-5)
@component(Cd2+, 1e-3)
```

The remaining statements in the input file can be placed in a "reaction" object class, as illustrated further below.

Example Input File Using Objects. The complete system can now be defined in terms of the component, species and reaction objects as follows:

```
@include("myobjects.txt")
@component(Cl-, 1e-5)
@component(Cd2+, 1e-3)
@species(CdCl+)
@reaction(CdCl+, 95.5, 1, Cd2+, 1 Cl-)
```

Note that because of the different calculation stages the reaction object can be defined after the species object. Some of the calculations involved with the species object occur after those of the reaction object.

The ORCHESTRA Object Database

The standard ORCHESTRA object database contains a collection of predefined model object classes. Similar to those shown above, these object class definitions are recursive, so that new object classes can be defined in terms of previously defined classes. This makes it possible to design a new object-oriented structure for implementing chemical models that consists of a set of three generic object classes (entity, reaction, and phase). These three object classes embody the important general properties of chemical model objects and their chemical interactions and serve as building blocks for the implementation of more specific model objects, such as "species", "components", "minerals", etc., that are similar to the abstractions used in standard programs.

The chemical model objects are currently defined in text format. However, literally the same structure could be used to implement the model definitions in a standard object-oriented computer language such as C++ or Java. Doing so would have the advantage that a standard computer language would be used for the model object definitions. However, this would require recompilation of the input file for each change in system definition. In the Java language, this can be implemented relatively easily, as separate modules (classes) can be compiled and dynamically linked.

Fundamental Chemical Object Classes. The three fundamental chemical object classes in the database from which all other model classes are composed are the *entity*, the *reaction*, and the *phase*.

Entity. The first fundamental object class is an "entity". An entity can be regarded as an abstract version of what is known as a "species" in standard models. In standard models, the calculations involved in a species are present in the source code of the calculation kernel. An ORCHESTRA entity is an object with an activity, (name.act), a concentration (name.con), and a mass balance in each system phase (name.phase.sum). Because these properties are shared by a wide range of chemical objects, such as aqueous species, gaseous species, electrons, precipitates, all types of surface species and surfaces sites, but also electrostatic layers, kinetic processes Donnan phases etc., these classes all have the same "entity" class as their parent class. The literal definition of the entity object class in the object database is

```
@Class: entity(name, phase){
  @Var: <name>.act 0
  @Var: <name>.con 0
  @Var: <name>.<phase>.sum 0
}
```

This version of entity contains just three variable definitions. A second version additionally defines a concentration variable and uses it to initialize its cumulative mass balance.

```
@Class: entity(name, phase, factor){
  @entity(<name>, <phase>)
  @Calc:(2, "<name>.con = <name>.act * <factor>")
  @Calc:(2, "<name>.<phase>.sum = <name>.con ")
}
```

Entities can be connected by reaction objects (see below) in a hierarchical way. This implies that as soon as an entity is defined or formed by a reaction it can subsequently be used in further reactions.

Reaction. The second fundamental object class is a reaction. A reaction represents the interaction between a number of entities and contains the calculations that embody their mass action, and mass balance relationships.

```
@Class: reaction(entity, k, coefficient_1, entity_1, ...,
                  Coefficient_n, entity_n){
  @Var: <entity>.k<k>
  @Calc:(1, "<entity>.act = <entity>.k")
  @link(<entity>, <entity_1>, <coefficient_1>)
  .....
  .....
  @link(<entity>, <entity_n>, <coefficient_n>)
}
```

The reaction object defines a reaction constant, *K*, for the

formed entity and initializes the activity of the entity with this *K* value during stage 1. The “link” objects that are used in the reaction object class have the same function as the stoichiometric coefficients in the traditional matrix approach. If two entities are “linked”, the code of the link object calculates the relationship between the activities/intensities and mass balances of both entities involved.

```
@Class: link(entity1, entity2, coefficient1, coefficient2){
  @Calc: (1,"<entity2>.act =
  <entity2>.act*<entity1>.act^<coefficient1>")
  @Calc: (3,"<entity1>.sum =
  <entity2>.sum+<entity1>.sum*<coefficient2>")
}
```

A link with a single stoichiometric coefficient uses this coefficient in the mass action as well as in the mass balance relationship.

```
@Class: link(entity1, entity2, coefficient){
  @link(<entity1>, <entity2>, <coefficient>,
  <coefficient>)
}
```

Phase. The third and final fundamental object class is a so-called “phase”. A phase represents a physical or logical phase of a system in which the mass balances of the entities are located. In terms of calculations, a phase object creates a distinct mass balance in this phase for each entity (<entity>.<phase>.sum). For example, the definition of the following phase object

```
@phase(total)
```

results in an extra mass balance for all entities with the name <entityname>.total.sum. Additionally, a statement is added to the link object to add the mass balance of the formed entity to that of the parent entity. Phases can be connected in a hierarchical way. A relationship between two phases indicates that the mass balances of entities in the progeny phase are part of the mass balance of the parent phase. A conversion factor between the two phases converts the mass balances of the daughter phase to the mass balance units of the parent phase. For example, the total phase of our example system could be composed of three daughter phases, including a water phase, a solid phase, and a gaseous phase:

```
@phase(water, total, watervolume)
@phase(gas, total, gasvolume)
@phase(solid, total, 1)
```

The mass balances of entities in the water, gas, and solid phases are now part of the mass balances in the total phase. During the calculations, the mass balances for each entity in the daughter phase are added to the mass balance in the total phase after multiplication by the conversion factor. Once a phase is defined, it can be subsequently used as a parent phase itself. For example, a colloidal phase that is part of the water phase can be defined as follows:

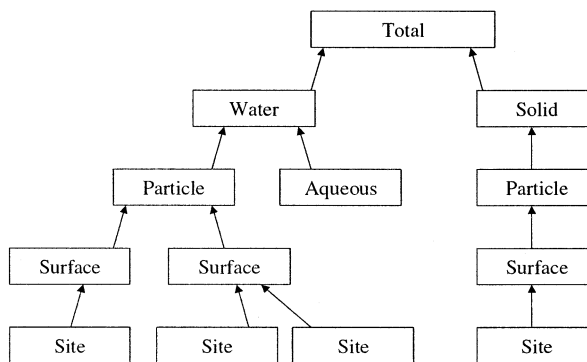
```
@phase(colloidal, water, colloid_mg/l)
```

The mass balances in the water phase now include the mass in the colloidal phase.

The resulting cumulative mass balances of entities per phase plus its daughter phases can be used as input for

(a)

Phase Hierarchy example



(b)

Entity Hierarchy example

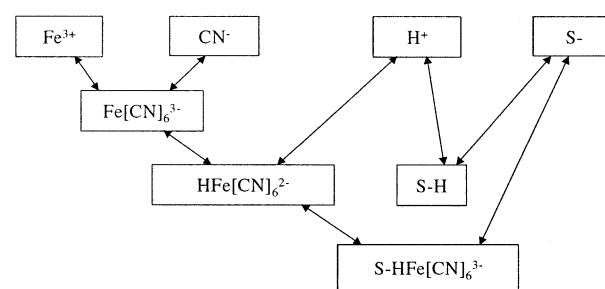


FIGURE 2. (a) Overview of the phase hierarchy in the example system. (b) Overview of the entity hierarchy in the example system.

transport or kinetic models. It is also possible to solve equilibrium systems for given mass balances in arbitrary system phases, e.g., the total dissolved, adsorbed, or gaseous amount of a component.

The ability to define distinct phases, with a chosen hierarchy, makes it relatively easy to implement systems with phases in which substances exhibit different transport behavior, such as solid, aqueous, gaseous, colloidal, or certain biological (microbial, plant root) phases. It is furthermore also possible to implement models that describe the chemical and transport behavior of a certain phase (e.g., adsorption and transport of colloids). Such models can be a function of any of the chemical parameters that are calculated by the rest of the model (surface charges, ionic strength, surface composition in terms of sites, etc.) An example of using ORCHESTRA for this purpose is described by Filius et al. (11).

Generally, only a minor fraction of the automatically generated mass balances will be used in output or during calculations. This, however, does not affect the calculation efficiency as the ORCHESTRA equation solver recognizes the redundant variables and calculations involved and removes these from the system.

Example. This example shows how the fundamental objects can be used to compose a simple chemical system, consisting of a set of five hierarchical phases, in which several entities are defined with accompanying formation reactions (Figure 2a.) The hierarchical definition of the phases, which contains a parent phase and a conversion factor between the mass balances of the daughter and parent phases, is as follows:

```

@phase(total)
@phase(water, total, water_volume)
@phase(particle, water, kg/l)
@phase(surface, particle, m2/g)
@phase(site, surface, mol/m2)

```

The units for the mass balances per phase are as follows: total phase, mol; water phase, mol/L; particle phase, mol/kg; surface phase, mol/m²; and in the site phase, mol/mol. The following entities are defined in the water and surface phases:

```

@entity(Fe3+, water)
@entity(CN-, water)
@entity(H+, water)
@entity(Fe(CN)63-, water)
@entity(HFe(CN)62-, water)
@entity(S-, surface)
@entity(S-H, surface)
@entity(S-HFe(CN)6, surface)

```

The formation reactions for all nonprimary entities are defined with the reaction object:

```

@reaction(Fe(CN)63-, k, 1, Fe, 6, CN-)
@reaction(HFe(CN)62-, k, 1, H+, 1, Fe(CN)63-)
@reaction(S-H, k, 1, H+, 1, S-)
@reaction(S-HFe(CN)6, k, 1, S-, 1, HFe(CN)62-)

```

This creates the entity hierarchy that is shown in Figure 2b. Note that reactions can be defined in terms of any previously defined entity, not just in terms of primary entities or components. Also, entities from different phases can be mixed in a formation reaction. Using an entity in a formation reaction automatically implies use of its activity to calculate the activity of the formed entity and lets the (cumulative) mass balance of the formed entity (times the stoichiometric coefficient) contribute to the mass balance of the reactants. For example, the mass balance of CN⁻ in the water phase results from the following calculations that are defined by the reaction objects:

```

HFe(CN)62-.water.sum =
  HFe(CN)62-.water.sum + 1 × S-HFe(CN)6.water.sum
Fe(CN)63-.water.sum =
  Fe(CN)63-.water.sum + 1 × HFe(CN)62-.water.sum
CN-.water.sum =
  1 × CN-.water.sum + 6 × Fe(CN)63-.water.sum

```

The first calculation is redundant, as S-HFe(CN)6.water.sum = 0 because this entity is located in the surface phase. However, it will be automatically removed. The entity hierarchy is replicated for each system phase. After the entity mass balances are calculated per phase, the phase hierarchy is subsequently used to sum the mass balances over the different phases.

For example, the mass balances of H⁺ in the different phases: H+.total.sum, H+.water.sum H+.particle.sum,

H+.surface.sum, and H+.site.sum, are calculated by traversing the phase hierarchy tree:

```

H+.surface.sum =
  H+.surface.sum + (mol/ m2) × H+.site.sum
H+.particle.sum =
  H+.particle.sum + (m2/g) × H+.surface.sum
H+.water.sum =
  H+.water.sum + (kg/l) × H+.particle.sum
H+.total.sum =
  H+.total.sum + (watervolume) × H+.water.sum

```

Example Model Implementations

It is now demonstrated how the fundamental object classes entity, reaction, and phase can be used as building blocks to construct a selection of standard equilibrium model types.

Species. A central abstraction in standard equilibrium programs is an (aqueous) species. A species has an activity and a mass balance and can be represented by an entity in the dissolved phase. Additionally, a species has a concentration variable and contains code that calculates the concentration from the activity. In this example, this is done according to the Davies activity correction model.

```

@Class: species(name, charge){
  @entity(<name>, dissolved)
  @calculate_concentration_with_davies(<name>,
                                       <charge>)
}

```

The “species” object class can be subsequently used to define an aqueous species in the input file:

```

@species(OH-, -1)
@reaction(OH-, 1e-14, -1, H+)

```

Because the “species” class is derived from the “entity” class, it can be used in a reaction. Note that, once a new entity is defined, it can itself be used in the definition of subsequent entities or species. OH⁻ can thus be used in the definition of a CaOH⁺ species:

```

@species(CaOH+, -1)
@reaction(CaOH+, 1e-10, 1, Ca2+, 1, OH-)

```

Component or Master Species. The second fundamental building block of chemical equilibrium systems is a so-called component or master species. This is effectively a species with a known activity or known total mass balance in a given phase. Because the activity of a component species is directly estimated as unknown during the iteration process, it does not require a reaction to calculate its activity.

```

@Class: component (name, charge, phase, value){
  @species(<name>, <charge>)
  @UnEq: (<name>.act, <name>.<phase>.sum,
          <value>)
}

```

This definition shows how arbitrary variables can be chosen as unknown-equation pairs. In this case, the activity of the species is declared as an unknown, and the mass balance in an arbitrary phase is used as the accompanying equation. The value of the unknown is found by iteration. A “component” with a given activity can be defined as a species

with this activity:

```
@Class: component_with_given_activity
      (name, charge, value){
    @Calc:(1,"<name>.act = <value>")
    @species(<name>, <charge>)
  }
```

Components, or primary entities, that are not based upon an aqueous species, such as electrons or surface components, can be included by using an entity instead of a species. To define chemical systems with electrons and redox reactions, it is sufficient to define an entity "e", present in the total phase. If the total amount of electrons in the system is known, the electron activity can be defined as an unknown and the total mass balance used as the accompanying equation:

```
@entity(e, total)
@UnEq: (e.act, 1e-8, e.total.sum, 1e-3)
```

Note that this definition gives the electron an activity and implements a cumulative mass balance that excludes the concentration of free electrons.

Implementation of Basic Chemical Equilibrium Models

Freundlich Adsorption Isotherm. The Freundlich adsorption isotherm is a widely used empirical model for soils. In this model, adsorbed ion concentrations are directly calculated from the dissolved concentrations as follows:

$$Q = KC^n$$

in which Q is the adsorbed amount, C is the dissolved concentration, K is the equilibrium constant, and n is the Freundlich coefficient. This model is not very complicated but requires two distinct stoichiometric coefficients in the mass balance and the mass action relationships. In ORCHESTRA, a Freundlich model can be composed of an entity with an accompanying formation reaction. The entity represents the adsorbed "species" and is located in the adsorbed phase. This entity is formed from a reaction with the dissolved species, with the Freundlich constant 0.7 as its mass action coefficient, and 1 as its mass balance coefficient.

```
@entity(Cd-ads, adsorbed, 1)
@reaction(Cd-ads, 1e-4, "0.7, 1", Cd2+)
```

Note that the adsorbing species can be any aqueous species and does not need to be a component or master species.

Implementations of Langmuir adsorption and ion exchange models can be found in the additional information.

Precipitation. Equilibrium with a precipitated phase can be taken into account with a "mineral" object that is defined as an entity in the solid phase. The "activity" of this entity contains, just as is the case for other entities, the ion activity product of the constituting ions times the formation constant (or divided by the solubility product). The saturation index is the logarithm of this activity. The saturation index is positive when the mineral is supersaturated, 0 when it is exactly in equilibrium with the solution, and negative when the solution is undersaturated.

In contrast with other entities or species, the amount of a mineral cannot be calculated directly from the ion activity product of the ions involved in the formation reaction. Therefore, the amount of mineral is directly estimated as an unknown. If positive, the value of the unknown variable is interpreted as the amount of the mineral, if negative as the logarithm of the undersaturation. The accompanying equation that should become zero during iteration depends on

the value of the unknown. If the unknown is positive, and mineral is present, the equation simply is the saturation index, which should become zero at equilibrium. If the unknown is negative, and no mineral is present, the equation that has to become zero is the difference between the estimated and the calculated saturation index.

The complete mineral object class is defined as follows:

```
@Class: mineral(name){
  // a mineral object is also an entity
  @entity(<name>, mineral)
  @Var: <name>.si 0
  @Var: <name>.unknown 0
  @Var: <name>.equation 0

  // If the unknown is positive it represents the
                                     amount of mineral
  @Calc:(1,"<name>.mineral.sum = if(<name>
                                     .unknown > 0, <name>.unknown, 0)")
  @Calc:(2,"<name>.si = log10(<name>.act)")
  @Calc:(2,"<name>.equation = <name>.si")

  // If unknown<0 subtract the unknown value from
  // the equation, so (saturation index - unknown = 0)
  @Stage:(2,"<name>.equation = <name>.equation -
  if(0 < <name>.unknown, 0, <name>.unknown)")
  @UnEq: (<name>.unknown, <name>.equation, 0)
}
```

A mineral object can now be used in an input file as follows:

```
@mineral (goethite)
@reaction(goethite, k, 1, Fe+3, 3, OH-)
```

Note that because the mineral object is derived from an entity, a standard reaction object can be used to represent the formation reaction.

Calculation Efficiency. The ORCHESTRA equation solver has a number of features to optimize the efficiency of the calculations. Most importantly, it stores the complete relationship between all input variables, calculations, and output variables. With information on which input variables are constant (during iterations), and which output variables are actually used, the solver optimizes the calculations by removing redundant calculations, replacing constant expressions, and simplifying operations such as multiplying by 0 or 1, taking numbers to the power of 1, etc. The equation solver also "knows" which part of the model must be recalculated when a subset of the input variables (e.g., a single-input variable during determination of partial derivatives) changes value. These optimizations are crucial for efficient calculation of chemical systems, as the majority of calculations, especially for larger systems, are either redundant or can be simplified. (Without these optimizations, calculations can easily take 1 order of magnitude longer.) Although modern compilers can, in principle, perform similar optimizations, the information on which variables are constant or used is not available when the source code is being compiled. This information is only available when the program is run for a certain system. The same level of efficiency will thus be difficult to obtain without the run-time optimizations that are implemented in ORCHESTRA. The only alternative is to hard code the complete problem in source code.

The equation solver is implemented in Java, a relatively new computer language (SUN, Microsystems, 1995) that runs

without recompiling on operating systems for which a Java virtual machine is available. Initially, Java programs were significantly slower than comparable C++ or FORTRAN programs; however, recent Java versions of ORCHESTRA run almost as efficiently as the original C++ version. Apart from being operating system-independent, another advantage of Java is that it contains standard language constructs for parallel processing, which has made it possible to design the ORCHESTRA calculation kernel to make efficient use of multiprocessor hardware. The following results show the run times for a multicomponent proton sulfate transport system on a dual-processor 500-MHz PC (12); the chemical system contains 10 (normal and surface) components, 28 species, 60 cells, a CD-MUSIC goethite surface, and a basic stern silica surface. The total number of chemical equilibrium calculations required for this transport calculation is ~45,000 (60 cells $\times$ 750 time steps).

ECOSAT(7) Borland C++ Windows NT 485 s	ORCHESTRA 1 processor Java (SUN JDK 1.3) 56 s	ORCHESTRA 2 processors Java (SUN JDK 1.3) 29 s
---	--	---

Using the ORCHESTRA "Equilibrium" Module within Other Algorithms. Coupling the ORCHESTRA chemical equilibrium module with other algorithms is relatively easy because (a) the equation solver is an independent "object" that reads a complete chemical definition from its own input file and (b) all variables that are used in the chemical calculations are available for input/output and can be overwritten/ used by the encapsulating program.

Using the ORCHESTRA "chemical equilibrium" subroutine requires just three (Java) statements in the parent algorithm:

```
Calculator equilibrium_calculator =
    new Calculator(inputfile);
equilibrium_calculator.set_io_variables
    (list_of_variable_names);
equilibrium_calculator.calculate(state_variables);
```

The first statement creates a (Java) calculator object that is "programmed" as a chemical equilibrium module by its own input file. The second statement defines the variables that are used for the exchange of information between the chemical equilibrium module and the parent algorithm during the calculations. The final statement shows how the chemical equilibrium calculator is used to perform a chemical equilibrium calculation for a given set of state variables.

This approach is used in the ORCHESTRA framework to build a mixing cell transport algorithm that allows users to define any number of grid cells, and arbitrary connections between pairs of cells and between cells and input, or output files and graphics. It has been used for 1d and 2d systems (12–14). It can be used in combination with the equilibrium module to calculate reactive transport. The most interesting aspect of this approach is that it allows users to define transport equations, in a similar way as is possible for chemical models, because the same calculator is used for the transport calculations. This also makes it possible to use the object database to define transport model objects. The

standard database contains objects for molecular diffusion and convection.

Acknowledgments

I am grateful to Willem van Riemsdijk, Edward Paterson, Vincent Meeussen, David Lumsdon, Wendy van Beinum, Nelleke Domburg, Jeanine Geelhoed, Jeroen Filius, and David Kinniburgh for their essential contributions and to three anonymous reviewers for their useful comments on an earlier version of the manuscript. This work was funded by the Scottish Executive Environment and Rural Affairs Department and the Dutch Ministry of Agriculture, Nature Management and Fisheries.

Supporting Information Available

Implementation of several more complex adsorption models. This material is available free of charge via the Internet at <http://pubs.acs.org>.

Literature Cited

- (1) Westall, J. C.; Zachara, J. L.; Morel, F. M. M. MINEQL, a computer program for the calculation of the chemical equilibrium composition of aqueous systems. Department of Civil Engineering, Massachusetts Institute of Technology Technical Note No. 18, 1972.
- (2) Allison, J. D.; Brown, D. S., and Novo-gradac, K. J. MINTEQA2, geochemical assessment model for environmental systems. Version 3. U.S. Environmental Protection Agency, Washington, DC, 1992.
- (3) Parkhurst, D. L.; Appelo, C. A. J., User's Guide to PHREEQC (Version 2), A Computer Program for Speciation, Batch-Reaction, One-Dimensional Transport, and Inverse Geochemical Calculations. U.S. Geological Survey, 2002.
- (4) The Geochemist's Workbench version 4.0 Rockware Earth Science Software, www.rockware.com.
- (5) Keizer, M. G.; Van Riemsdijk, W. H. ECOSAT, a Computer Program for the Calculation of Chemical Speciation and Transport in Soil–Water Systems, Wageningen Agricultural University, 1995.
- (6) Tipping, E. *Comput. Geosci.* **1994**, *20*, 973–1023.
- (7) Kinniburgh, D. G.; Riemsdijk, W. H.; Koopal, L. K.; Borkovec, M.; Benedetti, M. F.; Avena, M. J. *Colloids Surf., A* **1999**, *151*, 147–166.
- (8) Benedetti, M. F.; Milne, C. J.; Kinniburgh, D. G.; Van Riemsdijk, W. H.; Koopal, L. K. *Environ. Sci. Technol.* **1995**, *29* (2), 446–457.
- (9) Fish, W.; Dzombak, D. A.; Morel, F. M. M. *Environ. Sci. Technol.* **1986**, *20* (7), 675–683.
- (10) Hiemstra, T.; Van Riemsdijk, W. H.; Bolt, G. H. *J. Colloid Interface Sci.* **1989**, *133* (1), 91–104.
- (11) Filius, J. D.; Lumsdon, D. G.; Meeussen, J. C. L. *Geochim. Cosmochim. Acta* **2000**, *64* (1), 51–60.
- (12) Meeussen, J. C. L.; Scheidegger, A.; Kleikemper, J.; Borkovec, M.; Paterson, E.; Van Riemsdijk, W. H.; Sparks, D. L. *Environ. Sci. Technol.* **1999**, *33*, 3443–3450.
- (13) Van Beinum, W.; Meeussen, J. C. L.; Van Riemsdijk, W. H. *Environ. Sci. Technol.* **2000**, *34*, 4902–4907.
- (14) Lumsdon, D. G.; Filius, J. D.; Meeussen, J. C. L., Modelling Cd binding on goethite in the presence of humic substances. In Proceedings 10th International Meeting of the International Humic Substances Society, Toulouse, France. 24–28 July 2000, Vol. 1, pp 549–552.
- (15) Gaines, G. L.; Thomas, H. C., *J. Chem. Phys.* **1953**, *21*, 714–718.
- (16) Vanselow, A. P. *Soil Sci.* **1932**, *33*, 95–113.

Received for review February 21, 2002. Revised manuscript received December 1, 2002. Accepted December 16, 2002.

ES025597S